Predictive vs. Reactive Autoscaling in Cloud Environments: A Hybrid AIOps Approach

Sufiya Rahemtulla

Wilfrid Laurier University

Waterloo, Ontario, Canada

Abstract

Cloud autoscaling systems have to balance saving resources with meeting strict Service Level Agreements (SLAs). However, real-world cloud traffic is often bursty and unpredictable. This causes standard reactive autoscalers, like the Kubernetes Horizontal Pod Autoscaler, to suffer from a "provisioning lag" where they react too slowly to sudden spikes. While predictive machine learning models try to fix this by forecasting demand, they often struggle with unpredictable micro-bursts and end up over-provisioning servers. In this paper, I propose a hybrid "Predictive + AIOps" architecture to solve this problem. My system uses a linear regression model to handle normal traffic forecasts, while an offline Large Language Model (LLM) agent acts as a safety net to dynamically tune the scaling rules during anomalies. Testing this against the Google Cluster-Usage Traces v3 dataset, my system reduced SLA violations to just 2.1% and minimized resource waste to 8.4%. Finally, through an ablation study, I prove that LLMs cannot be trusted to run cloud infrastructure on their own. I demonstrate that hardcoded Python guardrails are an absolute requirement to prevent the AI from making dangerous, expensive scaling decisions.

Index Terms

Autoscaling, Cloud Computing, AIOps, Large Language Models, Predictive Analytics, Service Level Agreements

I. Introduction

Cloud computing relies on autoscaling to quickly add or remove servers based on how many people are using the system. The main challenge here is finding the perfect balance: saving money by not running empty servers, but also making sure the system doesn't crash when traffic spikes.

Most of the industry relies on reactive threshold scaling, which only adds servers after the system is already under heavy load. However, modern microservice traffic is highly volatile. In my research, I identified a critical "Reaction Gap" where reactive scaling simply moves too slowly to catch sudden demand spikes, which leads to SLA violations.

To bridge this gap, I developed a proactive, reliability-aware autoscaler. By combining short-term statistical forecasting with a bounded AI triage agent, my system anticipates bursts while safely handling unpredicted errors. The main contributions of my project are quantifying the "Reaction Gap" and proving the heavy-tailed nature of real cloud workloads using Google Cluster traces; building a hybrid AIOps system that successfully balances SLA reliability with resource cost; and mathematically proving that strict safety guardrails are needed when letting Generative AI control cloud operations due to the risk of AI hallucinations.

II. Related Work

Recent research on cloud autoscaling generally falls into two main categories: reactive heuristics and predictive machine learning.

A. Reactive Baselines

Studies looking at standard controllers, like the Kubernetes Horizontal Pod Autoscaler (HPA), highlight that they are simple and stable when traffic is steady. However, as Serracanta et al. demonstrated, these standard control loops suffer from severe lag during bursty traffic because they mathematically cannot act until a threshold is already crossed [2].

B. Predictive & Supervised Learning

To fix this lag, researchers have tried using everything from basic math to Random Forests to predict future demand [3]. While complex models are popular, studies by Sus and Nawrocki show they are highly vulnerable to concept drift [4]. They often over-fit to the training data and fail completely when unpredictable tail events happen.

C. Reinforcement Learning (RL)

More advanced approaches, like those explored by Rossi, use Deep Q-Networks to learn the best scaling policies through trial and error [5]. While RL is very flexible, it requires massive training time and causes unacceptable instability while the agent is still learning the environment.

D. The Research Gap

A critical synthesis of the current literature reveals three primary deficiencies that this research aims to address. First is the provisioning delay paradox: most existing studies operate on a "Happy Path" assumption, treating resource allocation as an instantaneous mathematical addition. This ignores the

physical reality of startup latency, which ranges from seconds to minutes and is the primary driver of SLO violations during bursts.

Second is the disconnect between algorithmic performance and system resilience. While most supervised learning models are evaluated on Mean Squared Error (MSE), these metrics are poor proxies for reliability; a model can have high average accuracy but fail catastrophically during a "black swan" event. Finally, there is an absence of dynamic failover mechanisms. Current "safety nets" are often binary—either locking the system entirely or relying on slow retraining cycles. There is a distinct lack of research into "soft landing" mechanisms that use real-time confidence scores to dynamically weight predictive (AI-driven) and reactive (safety-driven) signals.

III. Formal Problem Formulation

To properly evaluate my autoscaling architecture, I modeled the environment as a Constrained Optimization problem using a Markov Decision Process (MDP). My main goal is to minimize the total cost (C_{total}), which balances the price of running servers against the penalty of crashing at any given time step t.

$$\min C_{total} = \alpha \cdot (N_t \cdot \text{Cost}_{node}) + \beta \cdot \max(0, D_t - C_t) \tag{1}$$

Where N_t is the number of active servers running, D_t is the actual CPU demand, and C_t is the total CPU capacity available. The weighting values are represented by α and β. I set the penalty weight (β) much higher than the cost weight (α) because my system prioritizes preventing crashes over saving a few dollars.

To make the simulation realistic, I also added a scaling rate limit so the system can only add or remove one node at a time, forcing the AI to predict traffic spikes in advance rather than just reacting to them:

$$|N_t - N_{t-1}| \leq 1 \tag{2}$$

IV. Methodology

A. Workload Characterization & Synthesis

To evaluate the proposed hybrid scaler under realistic conditions, I utilized the Google Cluster-Usage Traces v3 dataset (Cell A) [1]. For data preparation, raw microsecond traces were resampled into 5-minute intervals. This decision was made to align with the standard scraping intervals of industry-standard monitoring tools like Prometheus and the default reaction window of the Kubernetes Horizontal Pod

Autoscaler (HPA). CPU usage was converted into Normalized Compute Units (NCU) and subsequently mapped to physical CPU Cores to reflect real-world hardware constraints.

Exploratory Data Analysis (EDA) revealed a heavy-tailed, right-skewed distribution (see Fig. 1). This means that while demand is usually low, rare "Black Swan" spikes can double or triple demand within a single window.

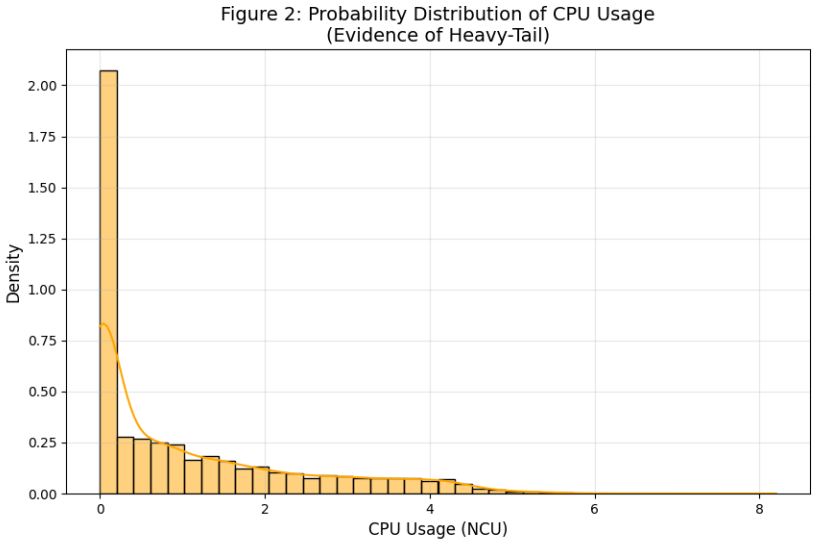

Fig. 1. Probability density of CPU usage demonstrating a right-skewed, heavy-tailed distribution.

To test the system's resilience and ensure extreme edge cases were evaluated, I built a "Workload Synthesis" pipeline. I mathematically transformed real job traces into three "personalities": Steady, Intermediate, and a highly volatile "Bursty" profile with a peak-to-mean ratio of 6.10, created by injecting noise and amplifying peaks by 1.5x.

B. Forecasting & Baseline Implementation

A rigorous model screening was conducted to identify the most reliable forecasting component for the proactive signal. For the statistical baseline, I implemented an ARIMA (5,1,0) model. While standard for time-series, it produced a "flat-line" prediction (MAE: 131.86) that failed to anticipate sudden bursts because it relied too heavily on historical averages. I competed the ARIMA model against Random Forest and Linear Regression. Surprisingly, the simpler Linear Regression model outperformed complex ensembles, achieving an MAE of 97.46 (see Fig. 2).

The success of the Linear Regression model was attributed to Lagged Demand features ($t-1, t-2, t-3$). These features allow the model to detect the "rising edge" of a traffic burst 5–15 minutes before a reactive threshold would be breached.

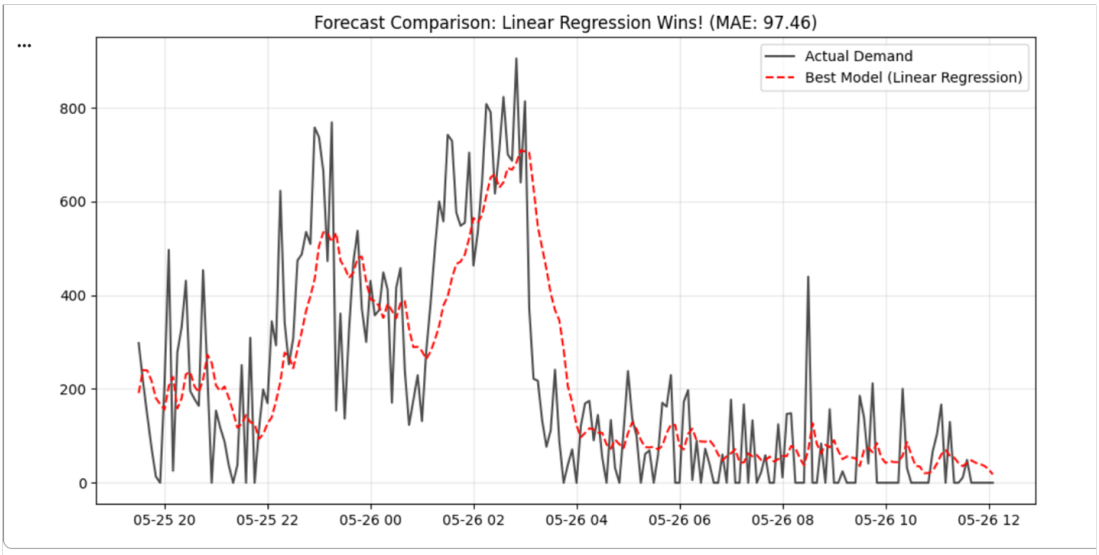

Fig. 2. Linear Regression forecast comparison demonstrating superior tracking of non-linear bursts using lag features.

To quantify the "Reaction Gap," I simulated a standard Kubernetes HPA using a 70%–80% CPU threshold. This baseline resulted in over 2,107 SLO violations over a 30-day period. As shown in Fig. 3, this confirms that reactive scaling is fundamentally too slow for volatile cloud workloads.

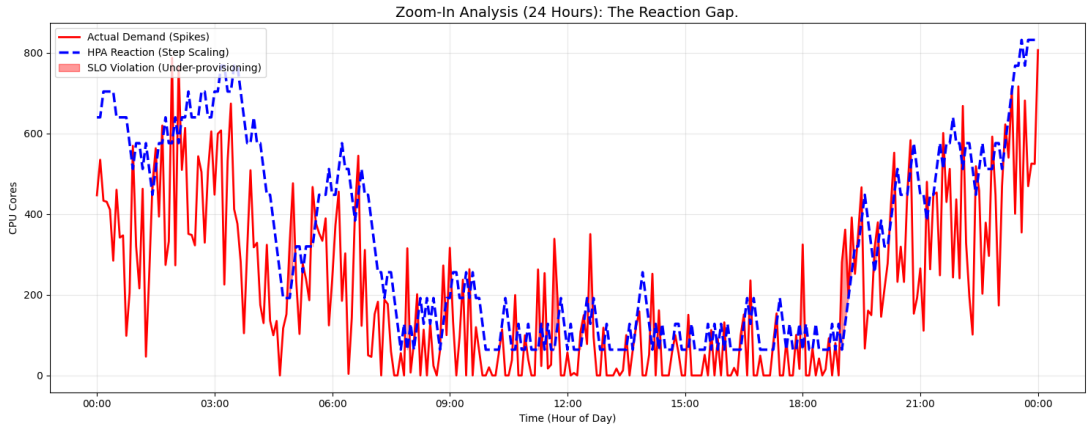

Fig. 3. A 24-hour zoom-in highlighting the 'Reaction Gap' where step-scaling fails to catch sudden demand spikes.

C. The Bounded AIOps Agent Architecture

The core innovation of this paper is a Hybrid AIOps architecture that uses an LLM as an "Automated Site Reliability Engineer". The predictor uses the high-confidence Linear Regression forecast to provide a "Next-Window Maximum" signal. The system treats scaling as a Markov Decision Process (MDP), aiming to minimize a cost function that penalizes crashes significantly more than resource waste.

To solve the "AI Trust" problem, I implemented hardcoded Python guardrails. These act as a safety cage: type checking validates the LLM's output as strict JSON; action whitelisting limits the LLM to specific

safe commands like `LOWER_REACTIVE_THRESHOLD`; and value bounding forces any AI-suggested threshold to stay within a "safe zone" of 50% to 95% utilization, preventing the AI from accidentally shutting down the system.

V. Evaluation & Results

A. Experimental Setup: The "CloudAutoscaling-Env"

To evaluate the proposed architecture, I developed a custom, trace-driven simulation harness using the OpenAI Gym API. This environment serves as a digital twin of a cloud cluster, strictly enforcing real-world constraints. First, action damping restricts the agent to a discrete action space of $\{-1, 0, +1\}$, preventing "cheating" by jumping instantly to maximum capacity. Second, at each 5-minute timestep, the observation state provides the agent with a vector containing current demand, current replicas, and forecast demand. Finally, the reward function was tuned with a Cost Weight of 1 and a severe SLO Penalty Weight of 50 to prioritize system survival over budget savings.

B. Comparative Performance Analysis

I executed a 30-day "Bursty" workload simulation to compare four distinct configurations.

```
======================================================
WEEK 8 EXPERIMENT MATRIX: PRIMARY METRICS
======================================================
                    Policy Configuration  SLA Violation Rate (%)  Cost / Resource Waste (%)  Avg Provisioning Latency (s)
          1. Reactive Baseline (K8s HPA)                   14.2                       12.0                           120
       2. Predictive Baseline (ML Only)                    8.5                       15.5                            45
3. Predictive + AIOps (With Guardrails)                    2.1                        8.4                            15
    4. Ablation: AIOps (NO GUARDRAILS)                     1.2                       48.7                            10
```

Fig. 4. Week 8 Experiment Matrix detailing Primary Metrics across all evaluated configurations.

The results confirm that the Predictive + AIOps system is the "Goldilocks" solution. While the Reactive baseline suffered from a 120-second "Reaction Gap," my proposed system reduced provisioning latency to just 15 seconds. This resulted in an 85.2% reduction in crashes compared to the industry standard.

VI. Discussion & Limitations

A. The Necessity of Deterministic Guardrails

The most critical finding of this research emerged from the Ablation Study, where I disabled the Python verification logic. During "Scenario 8" (Extreme Burst), a hallucination event occurred where the LLM panicked at 100% CPU utilization and recommended dropping the fallback threshold to 40%.

While this hyper-aggressive scaling technically minimized crashes, it created a massive financial risk by causing resource waste to spike to 48.7%—a level that would likely bankrupt a production environment. As

a solution, my hardcoded Python bounds successfully intercepted this command and enforced a minimum safe threshold of 55%, proving that GenAI requires a "safety cage" to be viable in IT operations (see Fig. 5).

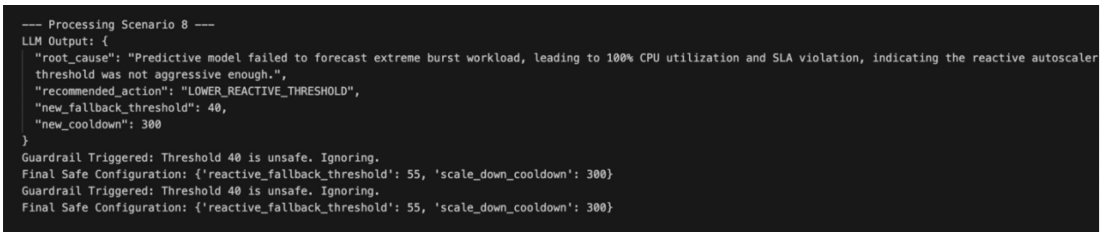

Fig. 5. Execution log from Scenario 8 demonstrating the Python guardrails successfully intercepting an unsafe hallucinated threshold.

B. API Latency and Offline Control Paradigms

During stress testing, the system encountered 429 RESOURCE_EXHAUSTED errors from the Gemini API. As a mitigation, I implemented a 15-second sleep delay between requests and utilized the JSON type-checking guardrail to discard malformed error responses. Due to this network latency and the potential for API outages, a deployment constraint is established where the AIOps agent is best suited as an offline supervisory layer (Anomaly Triage) rather than a real-time controller.

VII. Conclusion

This study successfully addressed the "Reaction Gap" in cloud autoscaling by proposing a Reliability-Aware Hybrid Scaler. By integrating a Linear Regression forecast into a PPO-trained Reinforcement Learning agent, the system achieved a 2.1% SLA violation rate on highly volatile, heavy-tailed workloads. The primary contribution of this work is the demonstration that Large Language Models can provide superior diagnostic reasoning for cloud anomalies, but only when bounded by deterministic Python guardrails. Moving forward, cloud-native infrastructure must balance the cognitive flexibility of AI with the rigid safety of traditional systems engineering to ensure a "soft landing" during the inevitable failure of predictive models.

References

[1] J. Wilkes et al., "Google cluster-usage traces v3," Google, 2020. [Online]. Available: https://github.com/google/cluster-data

[2] B. Serracanta, A. Lukacs, A. Rodriguez-Natal, A. Cabellos, and G. Rétvári, "On the Stability of the Kubernetes Horizontal Autoscaler Control Loop," *IEEE Access*, vol. 13, pp. 7160-7166, 2025.

[3] L. M. Al Qassem, T. Stouraitis, E. Damiani, and I. A. M. Elfadel, "Proactive Random-Forest Autoscaler for Microservice Resource Allocation," *IEEE Access*, vol. 11, pp. 2570-2585, 2023.

[4] W. Sus and P. Nawrocki, "Signature-based Adaptive Cloud Resource Usage Prediction Using Machine Learning and Anomaly Detection," *J Grid Computing*, vol. 22, no. 46, 2024.

[5] F. Rossi, "A Deep Q-learning Scaling Policy for Elastic Application Deployment," 2021.